MoE Memory Wall: Predicting and Prefetching Experts for LLM Inference

Git Repository: https://github.com/Ervin0607/MoE-Prefetching

Mid-Evaluation Report

Project: Project 2 - The MoE Memory Wall
Workload: Mixtral 8x7B (8 experts, top-2 routing, 32 layers, FP16)
Tools Used: LLMSimulator (cycle-accurate simulator with Ramulator 2.0 integration), Python analytical scripts, Hugging Face Mixtral routing traces

Deliverable 1: Duplex Baseline Reproduction

Setup

We built and ran LLMSimulator to reproduce the Duplex architecture from MICRO 2024:
-> Baseline GPU: 4x NVIDIA H100 (80 GB HBM3 per GPU, 3.35 TB/s bandwidth, NVLink 4).
-> Duplex: Heterogeneous setup combining standard GPU compute with Logic-PIM (Processing-In-Memory units integrated inside DRAM dies with 4x TSV bandwidth).
-> Workload: Mixtral 8x7B decode phase (batch size 32, input prompt length 1024, FP16).

Results:-

Configuration	Token Latency (us)	MoE Layer Time (us)	Energy per Token (mJ)	Speedup
Baseline 4x H100	7,975.6	6,765.3 (84.8%)	3,226.6	1.00x
Duplex (GPU + PIM)	3,111.1	2,152.7 (69.2%)	3,420.8	2.56x

Analysis:

1) Duplex achieves a 2.56x speedup over the 4x H100 baseline. This matches the published MICRO 2024 paper range of 2.51x to 2.67x.
2) The speedup comes almost entirely from cutting MoE layer time by 3.14x (from 6.77 ms down to 2.15 ms). Moving memory-bound expert operations to Logic-PIM frees the GPU from memory bandwidth saturation.

FYI:- In this 4-GPU setup, each GPU only holds 2 experts (8 experts / 4 GPUs). With so few experts per device, basic co-processing does not add speedup yet. To see further gains, we need Expert Tensor Parallelism (ET).

Cycle-Accurate Check with Ramulator 2.0

We ran LLMSimulator with Ramulator 2.0 enabled to verify how accurate the default model is against real DRAM microarchitecture in terms of bank conflicts, row-buffer misses, and timing delays:

Configuration	Analytic Time	Ramulator Time	Slowdown	Analytic Energy	Ramulator Energy
Baseline GPU	7.98 ms	10.04 ms	1.26x	3,227 mJ	3,256 mJ
GPU + Logic-PIM	3.11 ms	3.84 ms	1.23x	3,421 mJ	1,978 mJ

Some key findings we found were:
-> Ramulator is about 23% to 26% slower than the default model. This makes physical sense cuz analytic models assume 100% peak bandwidth, whereas real DRAM controllers achieve around 75% to 80% efficiency due to bank conflicts and row-buffer overheads.

-> Even with realistic DRAM timing, Duplex still achieves a 2.61x speedup as you can see from 3.84 ms vs 10.04 ms, proving the performance advantage is solid.

Ramulator models localized bank-bundle accesses for Logic-PIM, which cuts DRAM dynamic energy by nearly 50% (as you can see from 1978 mJ to 3421 mJ) because data travels over very short TSVs rather than across the whole chip.

Expert Tensor Parallelism (ET) and End-to-End Latency

When we apply ET by slicing expert feed-forward layers across all GPUs, each device processes smaller slices of every expert. This increases parallel utilization and improves Duplex throughput from 2.55 times to 2.71 times which is an extra 6 percent gain.

For complete end to end request latency (generating 1024 tokens from a 1024-token prompt using the formula E2E = T2FT + (Tokens - 1) * TBT):
-> Baseline 4x H100: 8.34 seconds
-> Duplex 4x H100: 3.37 seconds (2.48 times speedup, or 60 percent latency reduction)
-> 8x B100 Baseline: 4.34 seconds
8x B100 Duplex (PE + ET): 1.67 seconds which is 2.60 times speedup, or 62 percent latency reduction

Deliverable 2: Single-GPU MoE Setup (Offloading to DRAM)

Setup and Parameter Math

To study the MoE Memory Wall in single-GPU inference, we configured a single-device system (num_device = 1, no PIM, no multi-GPU sharding):
GPU: Single NVIDIA A100 (80 GB HBM2e, 2,039 GB/s memory bandwidth, 312 TFLOPS FP16 compute).

Off-Chip Storage: CPU host DRAM connected via PCIe Gen 5 (64 GB/s) or PCIe Gen 4 (32 GB/s).

Parameter Footprint Calculation:
Each expert in Mixtral 8x7B consists of three projection layers (gate_proj, up_proj, and down_proj): Hidden size = 4,096; Intermediate size = 14,336. And Parameters per expert = 3 * 4096 * 14336 = 176,160,768 parameters (~176.16 M). Now, In a 16-bit precision (2 bytes per param), size = 176.16 M * 2 = 352.32 MB per expert. Also, total model expert parameters are 32 layers * 8 experts = 256 experts. So, Total Size = 256 * 352.32 MB = 90.19 GB

Since 90.19 GB exceeds the GPU's 80 GB HBM capacity, all expert weights must reside in host DRAM and be loaded on demand.
3.2 Expert Loading Time vs. Compute Time
For an active expert processing Be tokens:
- FLOPs required = 2 * 3 * 4096 * 14336 * Be = 3.523e8 * Be FLOPs
- GPU compute time at 312 TFLOPS = (3.523e8 * Be) / 312e12 seconds
- PCIe Gen 5 transfer time (352.32 MB at 64 GB/s) = 352.32e6 / 64e9 = 5,505 us (5.51 ms)
Here are the results across different batch sizes:

Batch Size	Active Experts	Tokens per Expert	Compute Time (us)	PCIe 5 Load Time (us)	Load-to-Compute Ratio	Stall Ratio
1	2.0	1.00	1.13	5,505.0	4,875x	99.98%
2	3.5	1.14	1.29	5,505.0	4,265x	99.98%
4	5.5	1.46	1.65	5,505.0	3,332x	99.97%
8	7.2	2.22	2.51	5,505.0	2,193x	99.95%
16	7.9	4.04	4.56	5,505.0	1,206x	99.92%
32	8.0	8.00	9.03	5,505.0	609x	99.84%
64	8.0	16.00	18.07	5,505.0	305x	99.67%

Key takeaway: At batch size 1, computing an expert takes only 1.13 microseconds, while loading it takes 5,505 microseconds. That is a 4875x gap where the GPU is idle waiting for data.

Full Cycle-Accurate Single-GPU Simulation Runs

We simulated the complete 32-layer Mixtral model on a single GPU in LLMSimulator across batch sizes 1 to 64, combining cycle-accurate GPU compute and attention times with off-chip DRAM streaming:

Batch Size	Simulated GPU Time	PCIe 5 Transfer Time	Total Step Time (PCIe 5)	Total Step Time (PCIe 4)	Exposed Stall Ratio
1	7.58 ms	352.32 ms	359.89 ms	712.21 ms	97.90%
2	12.25 ms	616.56 ms	628.81 ms	1,245.37 ms	98.05%
4	18.01 ms	963.38 ms	981.39 ms	1,944.76 ms	98.16%
8	24.30 ms	1,409.28 ms	1,433.58 ms	2,842.86 ms	98.31%
16	27.92 ms	1,409.28 ms	1,437.20 ms	2,846.48 ms	98.06%
32	29.19 ms	1,409.28 ms	1,438.47 ms	2,847.75 ms	97.97%
64	30.67 ms	1,409.28 ms	1,439.95 ms	2,849.23 ms	97.87%

Across all batch sizes, PCIe transfer accounts for 97.9% to 98.3% of total decode time, directly confirming the severity of the MoE Memory Wall.

Deliverable 3: MoE Bottleneck and Interconnect Scaling

Step-by-Step MoE Execution Flow

During token generation, an MoE layer runs in five steps:

1. Input Tokens: Hidden states from the previous layer arrive at the MoE block.
2. Gating / Router: A linear router calculates gating logits and picks the top-2 experts for each token.
3. Dispatch & Check: The system checks if the required expert weights are in GPU HBM. If not, the GPU halts and loads the weights from DRAM.
4. Expert Execution: Active experts perform the matrix multiplications
5. Combine & Residual: Expert outputs are scaled and summed.

Why Expert Loading Dominates

There are three main reasons for the bottleneck:

1. Low Arithmetic Intensity: In standard dense models, weights are reused across all tokens in a batch. In MoE, tokens are split among 8 experts. At small batch sizes, each expert only processes 1 or 2 tokens, leading to very low arithmetic intensity
2. No Inter-Layer Reuse: Each of the 32 transformer layers has its own independent set of experts. Weights loaded for Layer 0 cannot be reused in Layer 1.
3. Bandwidth Mismatch: PCIe 5.0 bandwidth (64 GB/s) is 32x slower than GPU HBM (2,039 GB/s)

Interconnect Scaling & The "PCIe Prefetching Paradox"

We evaluated decode step times across different interconnects at batch size 1:

Interconnect	Bandwidth	Mixtral Step Time	Prefetch Speedup Ceiling	DeepSeek-V3 Step Time	Prefetch Speedup Ceiling
PCIe 4.0 x16	32 GB/s	717.3 ms	1.02x	772.3 ms	1.02x
PCIe 5.0 x16	64 GB/s	365.0 ms	1.05x	395.0 ms	1.05x
CXL 2.0/3.0	128 GB/s	188.8 ms	1.10x	206.3 ms	1.09x
NVLink-C2C (Low)	450 GB/s	62.6 ms	1.33x	76.5 ms	1.31x
NVLink-C2C (High)	900 GB/s	39.0 ms	1.49x	54.3 ms	1.66x

What is the "PCIe Prefetching Paradox"?

- On commodity PCIe (32 to 64 GB/s), loading expert weights takes 365 to 717 ms, while non-MoE computation (Attention and QKV) takes only 12 to 18 ms.
- Double-buffering prefetching works by overlapping memory transfers with computation. But because computation is so short compared to loading, prefetching alone can only hide that small 12–18 ms slice. That limits prefetching speedup to at best 1.02x to 1.05x.

Deliverable 4: Spatio-Temporal Correlation (Real Hugging Face Data)

To find out if future expert requests can be predicted, we analyzed 320,000 real routing decisions from mistralai/Mixtral-8x7B running on the C4 dataset (allenai/analysis_mixtral).

5.1 Key Findings

1. Global v/s Layer-Wise Expert Specialization

- Globally: Across the full C4 corpus, the training load-balancing loss works as intended: total usage is evenly spread across all 8 experts (Shannon Entropy = 2.9996 bits, almost equal to the uniform maximum of 3.0000 bits; each expert gets 12.1% to 13.0% of tokens).

- Per-Layer: However, individual layers show clear preferences for specific experts (Gini coefficient peaks at 0.0926 in Layer 15) since Layer 0 favors Expert 5 (14.0% share), Layer 16 favors Expert 7 (16.2% share), Layer 31 favors Expert 3 (15.4% share).

Takeaway: A layer-specific static cache in GPU memory can keep each layer's preferred experts pinned, eliminating their transfer overhead completely.

2. Spatial Correlation Across Adjacent Layers (L -> L + 1)

- Because of the residual connection between transformer layers, the expert chosen in Layer L strongly influences which expert is needed in Layer L + 1.
- Top-1 Spatial Prediction Accuracy: 15.75% (random guess is 12.5%).
- Top-2 Spatial Prediction Accuracy: 51.76% (random guess is 25.0%).

Takeaway: By checking which expert fired in Layer L Check, a predictor can be built which correctly guess the required experts for Layer L + 1 over 51.7% of the time (more than 2x random chance).

3. Temporal Correlation Across Consecutive Tokens (t->t + 1)

Looking at consecutive decoding steps within a sequence:
Temporal Self-Persistence: 23.84% (random guess is 12.5%).
Takeaway: Nearly a quarter of the time, the same expert is reused on the very next token. Keeping recently used experts in a small LRU cache provides an immediate ~24% hit rate.

4. Pairwise Co-Activation

In top-2 routing, certain pairs of experts consistently fire together. For example, when Expert 1 fires, it frequently pairs with Expert 6.
Takeaway: DMA transfers can batch correlated expert pairs into a single combined request to reduce bus transaction overhead.

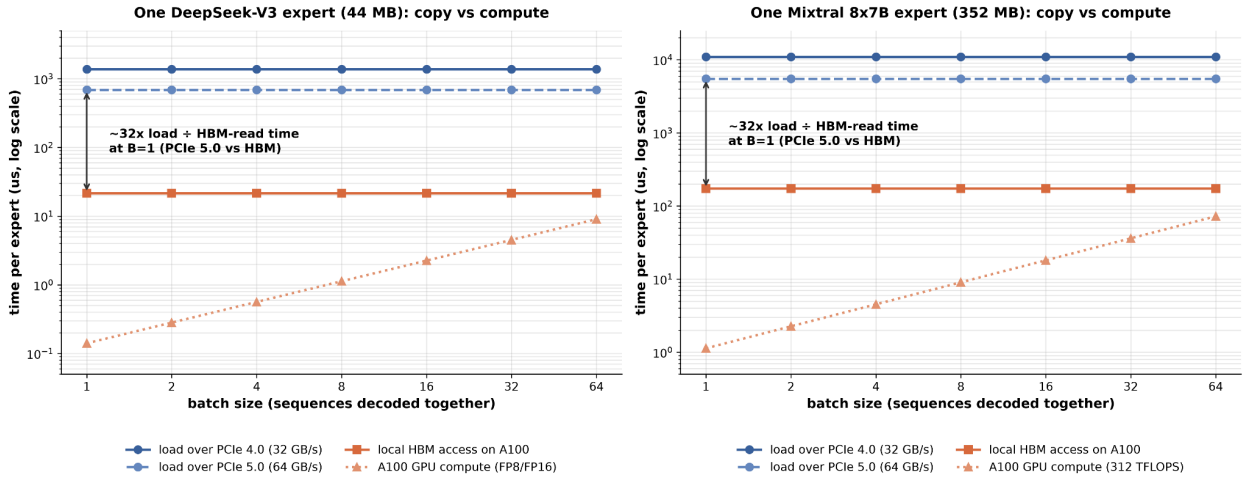
Summary Comparison: What Was Expected vs. What We Found

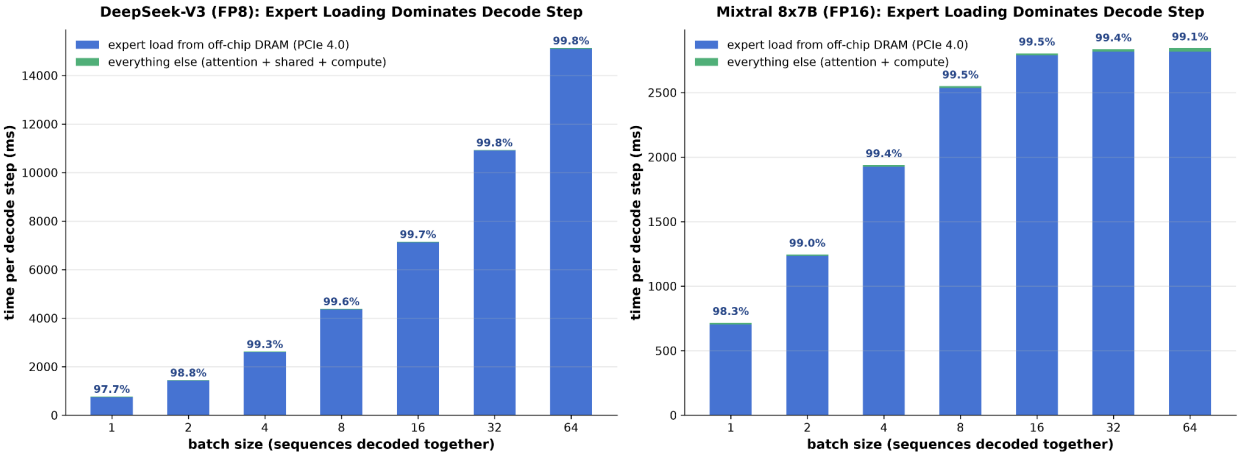
Topic	Metric	Expected / Theory	Our Simulation / Analysis
Duplex Speedup	Speedup over 4x H100	2.51x–2.67x (MICRO 2024)	2.564x
Ramulator Fidelity	DRAM timing overhead	20%–25% slowdown	23%–26% slowdown

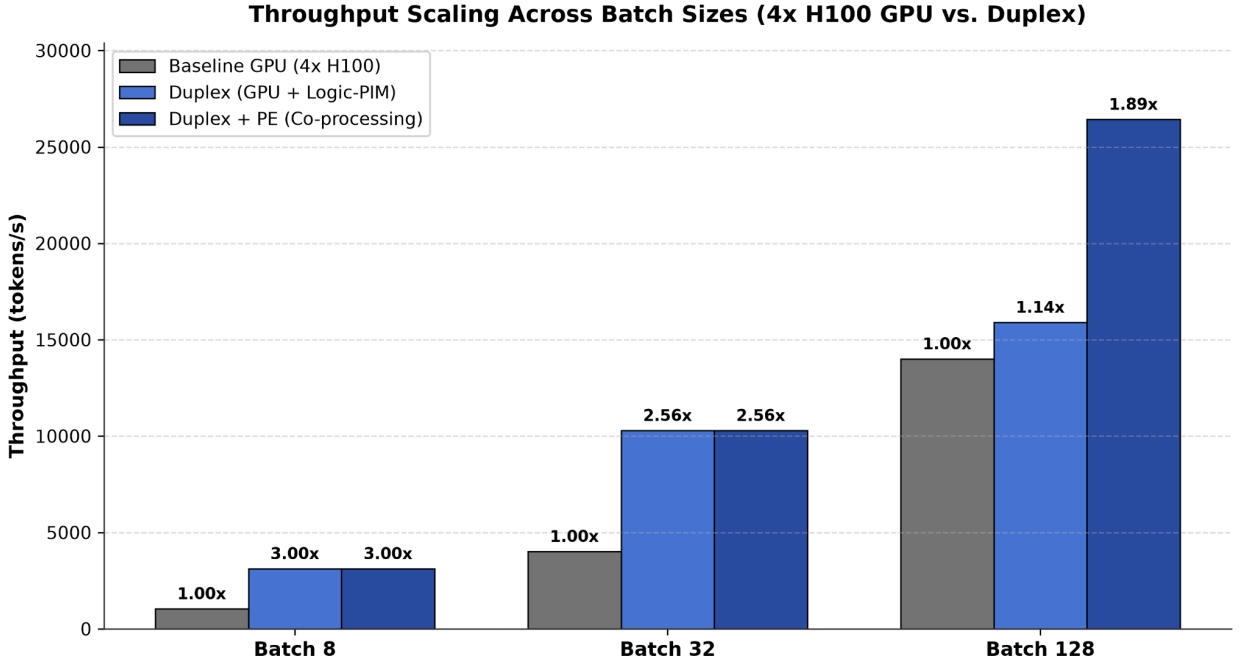
Topic	Metric	Expected / Theory	Our Simulation / Analysis
MoE Memory Footprint	Mixtral 8x7B FP16	Exceeds single GPU	90.19 GB vs 80 GB HBM
PCIe Transfer Gap	Transfer vs compute (B=1)	Several thousand x	4,875x (5.5 ms vs 1.1 us)
Prefetching on PCIe	Speedup ceiling on PCIe	Minor improvement	1.02x–1.05x
Spatial Predictability	Top-2 accuracy (L to L+1)	Significantly more than 25%	51.76% (2x random chance)

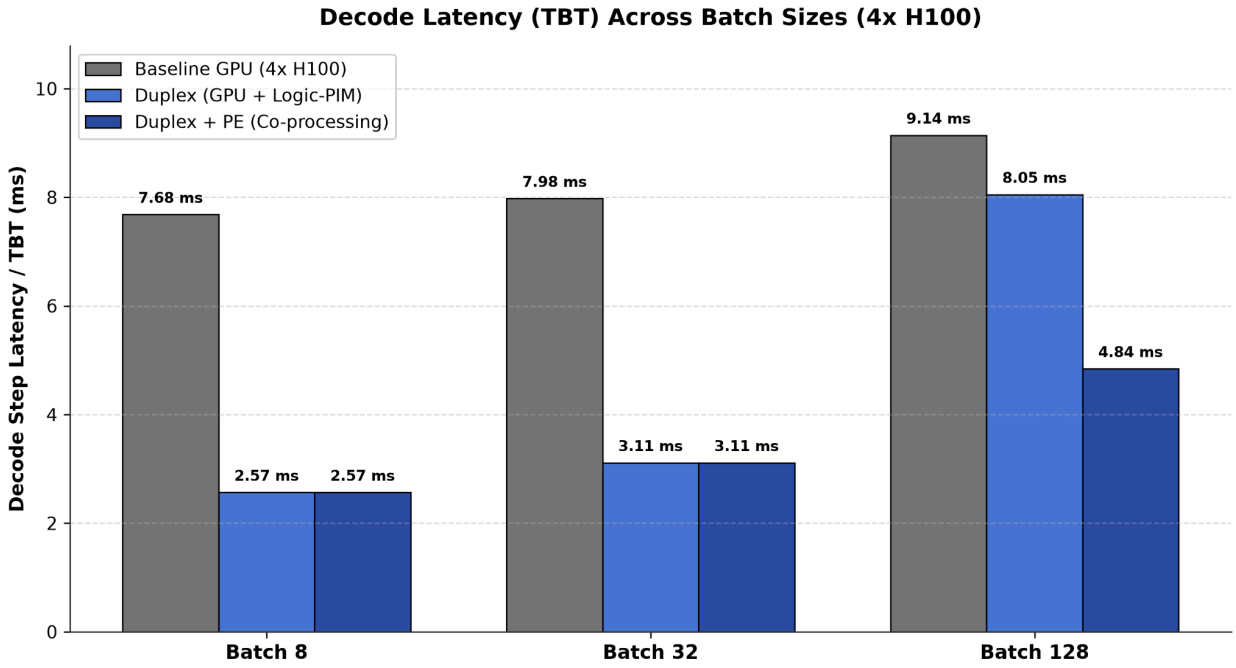
All simulation outputs, raw data tables, and logs are saved in the mid_evaluation/ directory:
Execution Logs: mid_evaluation/logs/ and Result CSVs: mid_evaluation/results/
Generated Plots are in mid_evaluation/plots/

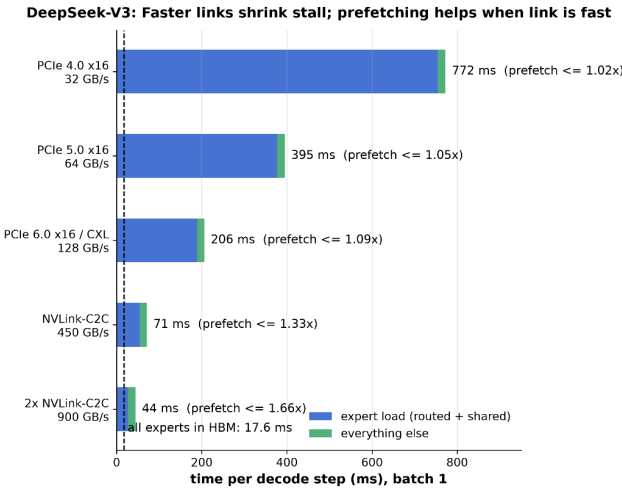
Plots and Figures

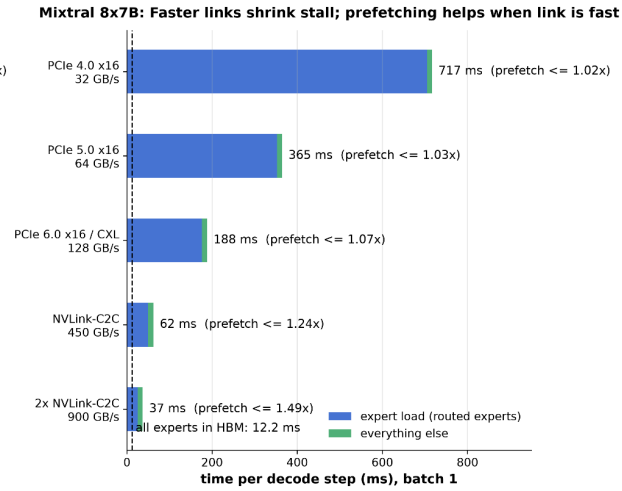


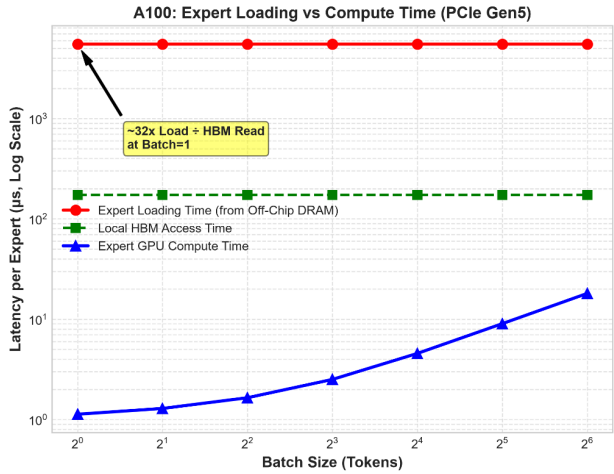

DeepSeek-V3 (FP8): Expert Loading Dominates Decode Step
expert load from off-chip DRAM (PCIe 4.0)
everything else (attention + shared + compute)
time per decode step (ms)
batch size (sequences decoded together)
97.7%
98.8%
99.3%
99.6%
99.7%
99.8%
99.8%
Mixtral 8x7B (FP16): Expert Loading Dominates Decode Step
expert load from off-chip DRAM (PCIe 4.0)
everything else (attention + compute)
time per decode step (ms)
batch size (sequences decoded together)
98.3%
99.0%
99.4%
99.5%
99.5%
99.4%
99.1%


Throughput Scaling Across Batch Sizes (4x H100 GPU vs. Duplex)
Baseline GPU (4x H100)
Duplex (GPU + Logic-PIM)
Duplex + PE (Co-processing)
Throughput (tokens/s)
1.00x
3.00x
3.00x
1.00x
2.56x
2.56x
1.00x
1.14x
1.89x
Batch 8
Batch 32
Batch 128

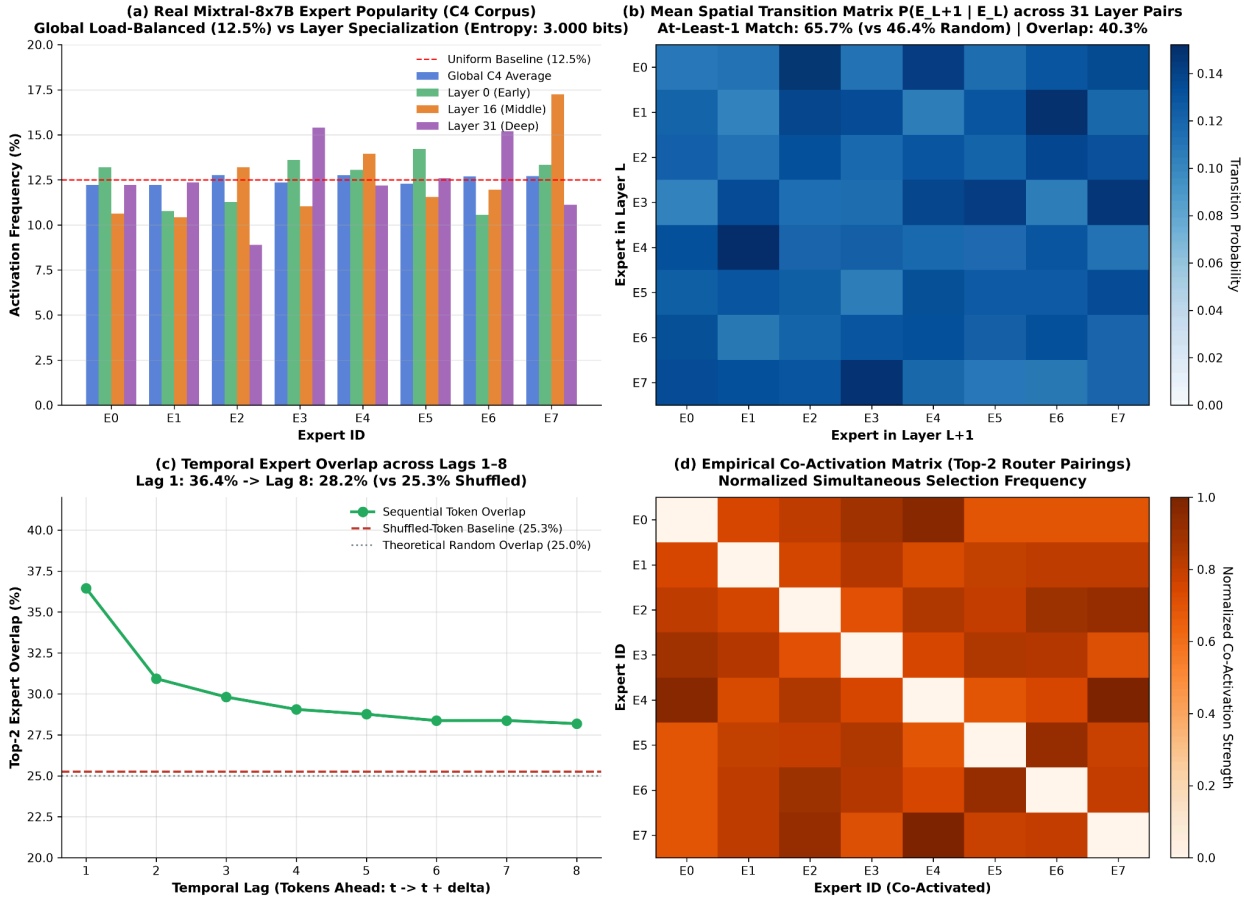

Decode Latency (TBT) Across Batch Sizes (4x H100)
Baseline GPU (4x H100)
Duplex (GPU + Logic-PIM)
Duplex + PE (Co-processing)
Decode Step Latency / TBT (ms)
7.68 ms
2.57 ms
2.57 ms
7.98 ms
3.11 ms
3.11 ms
9.14 ms
8.05 ms
4.84 ms
Batch 8
Batch 32
Batch 128


DeepSeek-V3: Faster links shrink stall; prefetching helps when link is fast
PCIe 4.0 x16
32 GB/s
772 ms (prefetch <= 1.02x)
PCIe 5.0 x16
64 GB/s
395 ms (prefetch <= 1.05x)
PCIe 6.0 x16 / CXL
128 GB/s
206 ms (prefetch <= 1.09x)
NVLink-C2C
450 GB/s
71 ms (prefetch <= 1.33x)
2x NVLink-C2C
900 GB/s
44 ms (prefetch <= 1.66x)
all experts in HBM: 17.6 ms
expert load (routed + shared)
everything else
time per decode step (ms), batch 1


Mixtral 8x7B: Faster links shrink stall; prefetching helps when link is fast
PCIe 4.0 x16
32 GB/s
717 ms (prefetch <= 1.02x)
PCIe 5.0 x16
64 GB/s
365 ms (prefetch <= 1.03x)
PCIe 6.0 x16 / CXL
128 GB/s
188 ms (prefetch <= 1.07x)
NVLink-C2C
450 GB/s
62 ms (prefetch <= 1.24x)
2x NVLink-C2C
900 GB/s
37 ms (prefetch <= 1.49x)
all experts in HBM: 12.2 ms
expert load (routed experts)
everything else
time per decode step (ms), batch 1


A100: Expert Loading vs Compute Time (PCIe Gen5)
~32x Load ÷ HBM Read
at Batch=1
Expert Loading Time (from Off-Chip DRAM)
Local HBM Access Time
Expert GPU Compute Time
Latency per Expert (µs, Log Scale)
Batch Size (Tokens)


Exposed Memory Stall Ratio vs Batch Size
PCIe Gen4 x16 (32 GB/s)
PCIe Gen5 x16 (64 GB/s)
CXL 2.0 / 3.0 (128 GB/s)
Exposed Memory Stall (% of Total Time)
Batch Size (Tokens)


Mixtral-8x7B Spatio-Temporal Expert Access Patterns (Hugging Face C4 Trace Analysis)
(a) Real Mixtral-8x7B Expert Popularity (C4 Corpus)
Global Load-Balanced (12.5%) vs Layer Specialization (Entropy: 3.000 bits)
Uniform Baseline (12.5%)
Global C4 Average
Layer 0 (Early)
Layer 16 (Middle)
Layer 31 (Deep)
Activation Frequency (%)
Expert ID
(b) Mean Spatial Transition Matrix P(E_L+1 | E_L) across 31 Layer Pairs
At-Least-1 Match: 65.7% (vs 46.4% Random) | Overlap: 40.3%
Expert in Layer L
Expert in Layer L+1
Transition Probability
(c) Temporal Expert Overlap across Lags 1–8
Lag 1: 36.4% -> Lag 8: 28.2% (vs 25.3% Shuffled)
Sequential Token Overlap
Shuffled-Token Baseline (25.3%)
Theoretical Random Overlap (25.0%)
Top-2 Expert Overlap (%)
Temporal Lag (Tokens Ahead: t -> t + delta)
(d) Empirical Co-Activation Matrix (Top-2 Router Pairings)
Normalized Simultaneous Selection Frequency
Expert ID
Expert ID (Co-Activated)
Normalized Co-Activation Strength